The `:allow_nil` option skips the validation when the value being validated is `nil`.

```
class Coffee < ActiveRecord::Base
  validates :size, inclusion: { in: %w(small medium large),
    message: "%{value} is not a valid size" }, allow_nil: true
end
```

3.2 :allow_blank

The `:allow_blank` option is similar to the `:allow_nil` option. This option will let validation pass if the attribute's value is `blank?`, like `nil` or an empty string for example.

```
class Topic < ActiveRecord::Base
  validates :title, length: { is: 5 }, allow_blank: true
end

Topic.create(title: "").valid?  # => true
Topic.create(title: nil).valid? # => true
```

3.3 :message

As you've already seen, the `:message` option lets you specify the message that will be added to the `errors` collection when validation fails. When this option is not used, Active Record will use the respective default error message for each validation helper.

3.4 :on

The `:on` option lets you specify when the validation should happen. The default behavior for all the built-in validation helpers is to be run on save (both when you're creating a new record and when you're updating it). If you want to change it, you can use `on: :create` to run the validation only when a new record is created or `on: :update` to run the validation only when a record is updated.

```
class Person < ActiveRecord::Base
  # it will be possible to update email with a duplicated value
  validates :email, uniqueness: true, on: :create

  # it will be possible to create the record with a non-numerical age
  validates :age, numericality: true, on: :update

  # the default (validates on both create and update)
  validates :name, presence: true
end
```

4 Strict Validations

You can also specify validations to be strict and raise `ActiveModel::StrictValidationFailed`when the object is invalid.

```
class Person < ActiveRecord::Base
  validates :name, presence: { strict: true }
end

Person.new.valid?  # => ActiveModel::StrictValidationFailed: Name
can't be blank
```

There is also an ability to pass custom exception to `:strict` option.

```
class Person < ActiveRecord::Base
  validates :token, presence: true, uniqueness: true, strict:
TokenGenerationException
end

Person.new.valid?  # => TokenGenerationException: Token can't be blank
```